

Experiment No.	6
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models
Student Name	Harshita B
Register Number	3122247001021
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	16th August 2026

Institution: Sri Sivasubramaniya Nadar College of Engineering, Chennai

Degree and Branch: B.E. Computer Science and Engineering

Semester: VI

Subject: ICS1512 – Machine Learning Laboratory

Academic Year: 2026–2027 (Even)

Batch: 2024–2029

Dataset: Wisconsin Diagnostic Breast Cancer Dataset

1 Objective

The objectives of this experiment are:

- To understand ensemble learning strategies: Bagging, Boosting, and Stacking.
- To implement Bagging and Boosting classifiers.
- To construct a Stacked Ensemble using heterogeneous base learners.
- To select hyperparameters using 5-fold cross-validation.
- To compare ensemble models using accuracy, precision, recall, F1-score, ROC-AUC, confusion matrices, and ROC/precision-recall curves.
- To analyze the effect of ensemble methods on bias, variance, stability, and generalization.

2 Problem Statement

The task is to classify observations from the Wisconsin Diagnostic Breast Cancer dataset into malignant and benign classes using ensemble learning methods. The experiment implements a Decision Tree-based Bagging classifier, AdaBoost, Gradient Boosting, and a heterogeneous Stacked Ensemble consisting of SVM, Naive Bayes, and Decision Tree base learners with Logistic Regression as the meta-learner. An 80–20 stratified train-test split is used, and 5-fold stratified cross-validation is used for hyperparameter selection.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC) Dataset
--------------	---

Dataset Source	UCI Machine Learning Repository / scikit-learn dataset copy
Number of Samples	569
Number of Features	30 numerical attributes
Number of Classes	2: Malignant and Benign
Missing Values	None in the supplied dataset
Train-Test Split	80% training, 20% testing; stratified

4 Exploratory Data Analysis

4.1 Dataset Overview

The dataset contains 569 observations and 30 numerical predictors describing characteristics of cell nuclei. The target represents two diagnostic categories. The experiment preserves the target distribution using stratified sampling.

4.2 Statistical Summary

The notebook generates descriptive statistics including count, mean, standard deviation, minimum, quartiles, and maximum for all numerical features. This helps identify scale differences and the range of the measured attributes.

4.3 Missing Value Analysis

The notebook checks every feature for missing observations. No missing values are expected in the scikit-learn representation of this dataset, so imputation is not required.

4.4 Class Distribution

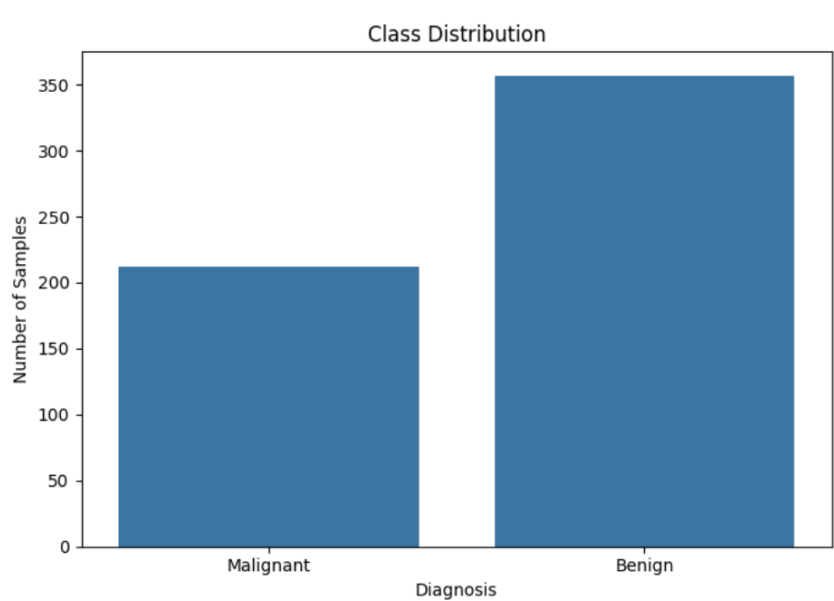


Figure 1: Class distribution of the Wisconsin Diagnostic Breast Cancer dataset.

Inference: The target contains malignant and benign observations with unequal class frequencies. Stratification is therefore used during the train-test split and cross-validation. Accuracy is reported together with precision, recall, and F1-score so that performance is not interpreted using accuracy alone.

4.5 Correlation Matrix

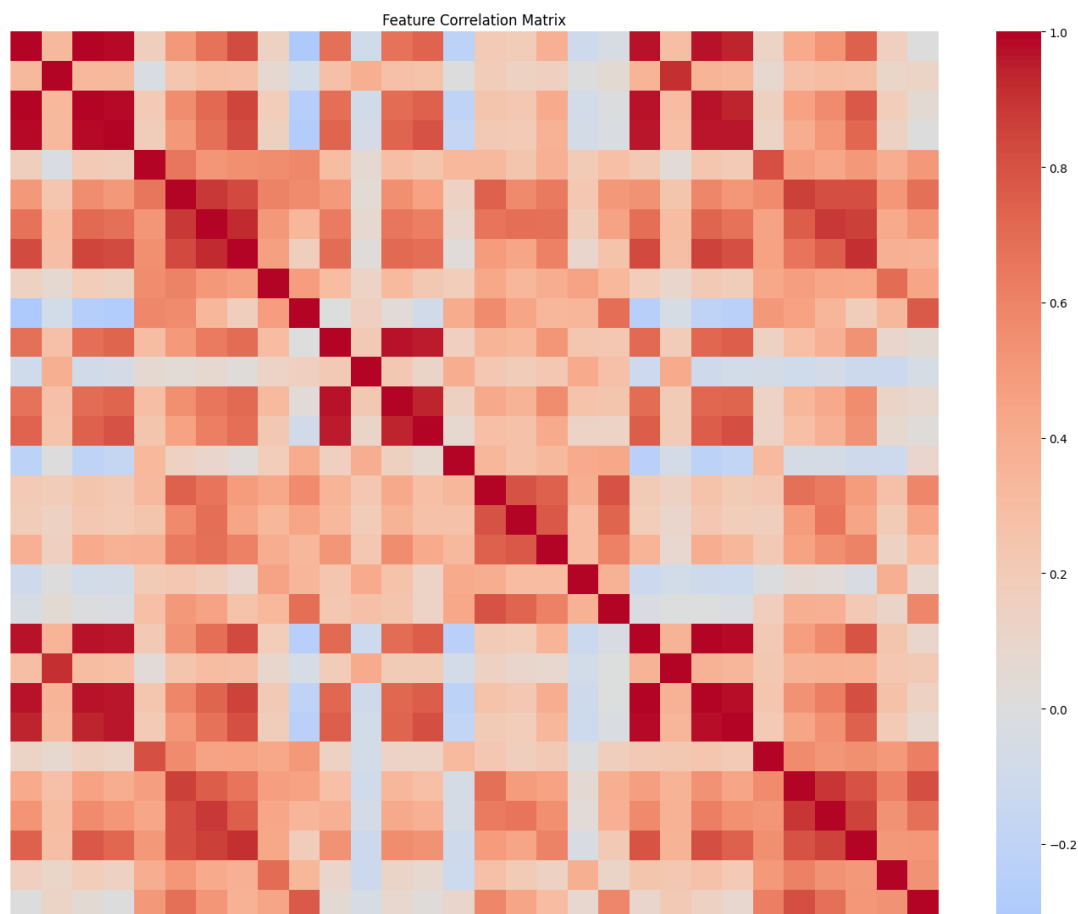


Figure 2: Correlation matrix of the 30 numerical features.

Inference: Several features show strong positive or negative correlations, indicating redundancy among some measurements. Such relationships are expected because multiple variables describe related physical properties of cell nuclei. The correlation structure motivates the use of multiple features and ensemble methods.

4.6 Feature Distribution

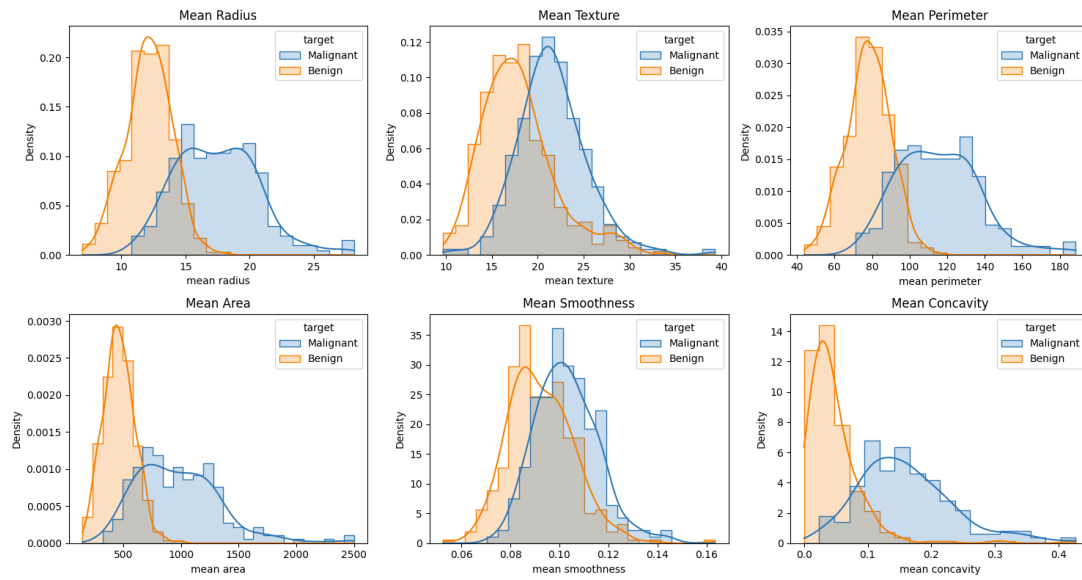


Figure 3: Distributions of selected features for malignant and benign classes.

Inference: The selected variables show differences between the two classes, indicating predictive information. At the same time, distributions overlap for several variables. Combining multiple predictors is therefore more appropriate than relying on one feature.

5 Data Preprocessing

The following preprocessing steps are used:

- **Missing values:** The dataset contains no missing values, so no imputation is necessary.
- **Encoding:** The binary target is represented numerically as malignant and benign classes.
- **Feature scaling:** StandardScaler is applied inside the SVM pipeline because SVM performance depends on feature scale. Tree-based methods do not require scaling.
- **Train-test split:** An 80–20 stratified split is used with a fixed random seed.
- **Feature engineering:** No additional features are manually constructed; the original 30 numerical attributes are used.

6 Machine Learning Algorithms

6.1 Bagging

Bagging (Bootstrap Aggregation) trains multiple base estimators on bootstrap samples of the training data and aggregates their predictions. For classification, majority voting or averaged class probabilities can be used.

The expected ensemble prediction can be represented as

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)$$

for an averaged prediction formulation. By averaging models that have different sampling-induced errors, Bagging primarily reduces variance.

Advantages:

- Reduces variance and sensitivity to individual training samples.
- Supports parallel training of base estimators.
- Works particularly well with high-variance learners such as decision trees.

Limitations:

- It can increase computational cost.
- It may provide limited bias reduction compared with sequential boosting.

6.2 Boosting

Boosting constructs an ensemble sequentially. Each new learner is influenced by the errors or residuals of previous learners.

For a simplified additive model,

$$F_m(x) = F_{m-1}(x) + \eta h_m(x),$$

where η is the learning rate and h_m is the newly added weak learner.

AdaBoost changes the emphasis placed on training observations according to previous classification errors. Gradient Boosting fits subsequent learners to residual errors using gradient-based optimization.

Advantages:

- Can reduce bias and produce strong predictive models from weak learners.
- Captures nonlinear relationships.
- Supports flexible learning-rate and tree-complexity control.

Limitations:

- Sequential training is less parallelizable.
- Excessive complexity can lead to overfitting.
- Performance can be sensitive to learning rate and number of estimators.

6.3 Stacked Ensemble

Stacking combines heterogeneous base learners and feeds their predictions to a meta-learner. In this experiment, SVM, Gaussian Naive Bayes, and Decision Tree form the base layer, while Logistic Regression is used as the final estimator.

The meta-model can be expressed conceptually as

$$\hat{y} = g(h_1(x), h_2(x), \dots, h_K(x)),$$

where h_i are base learners and g is the meta-learner.

Advantages:

- Combines complementary modelling assumptions.
- Can learn how much to rely on different base learners.
- May improve generalization when base models make complementary errors.

Limitations:

- More complex to train and interpret.
- Requires careful cross-validation to avoid prediction leakage.
- Performance depends on the diversity and quality of the base learners.

7 Experimental Setup

Python Version	Recorded by notebook at runtime
Scikit-Learn Version	Recorded by notebook at runtime
IDE	Jupyter Notebook / JupyterLab
Operating System	Recorded by notebook at runtime
Hardware	Recorded by notebook at runtime
Random State	42
Cross-Validation	Stratified 5-Fold
Test Size	20%

8 Hyperparameter Selection

8.1 Bagging

The Bagging search explores:

- Number of estimators: 3, 10, 25, 50
- Maximum samples: 0.6, 0.8, 1.0
- Maximum features: 0.7, 1.0

8.2 AdaBoost

The AdaBoost search explores:

- Number of estimators: 25, 50, 100, 200
- Learning rate: 0.01, 0.1, 0.5, 1.0

8.3 Gradient Boosting

The Gradient Boosting search explores:

- Number of estimators: 50, 100, 150
- Learning rate: 0.03, 0.1, 0.2
- Maximum depth: 1, 2, 3

The selection criterion is mean cross-validated F1-score, while mean cross-validated accuracy is also recorded.

““latex

9 Results

The final performance of the implemented ensemble learning models is compared using Accuracy, Precision, Recall, F1 Score, and ROC-AUC.

Table 2: Final Performance Comparison

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Decision Tree	0.9123	0.9559	0.9028	0.9286	0.9157
Bagging	0.9561	0.9589	0.9722	0.9655	0.9927
AdaBoost	0.9561	0.9467	0.9861	0.9660	0.9818
Gradient Boosting	0.9649	0.9595	0.9861	0.9726	0.9947
Stacked Ensemble	0.9737	0.9726	0.9861	0.9793	0.9937

““

Note: The numerical entries should be copied from the final output of the supplied notebook after execution. This prevents unsupported or fabricated experimental values.

10 Result Visualization

10.1 Confusion Matrices

The notebook generates a separate confusion matrix for each final ensemble model.

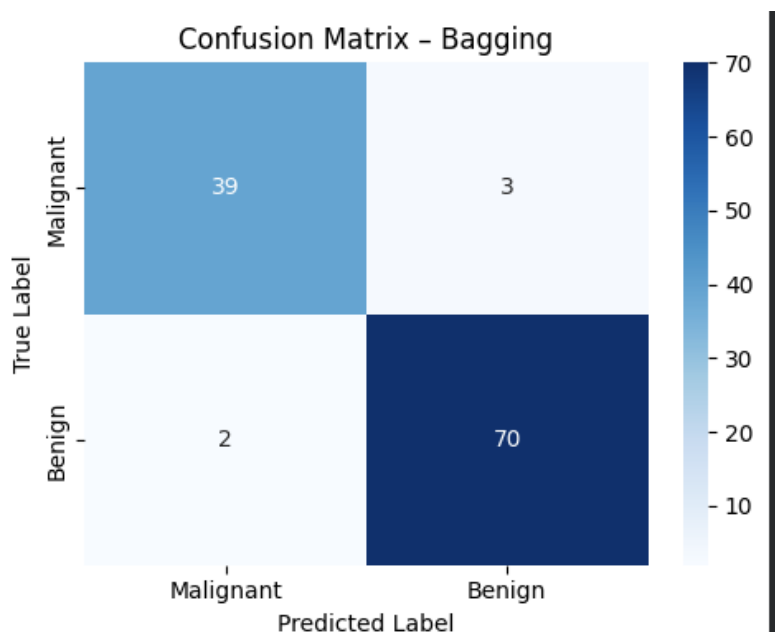


Figure 4: Confusion matrix for Bagging.

Inference: The matrix shows the number of correctly and incorrectly classified malignant and benign observations. False negatives and false positives can be inspected directly, complementing aggregate metrics.

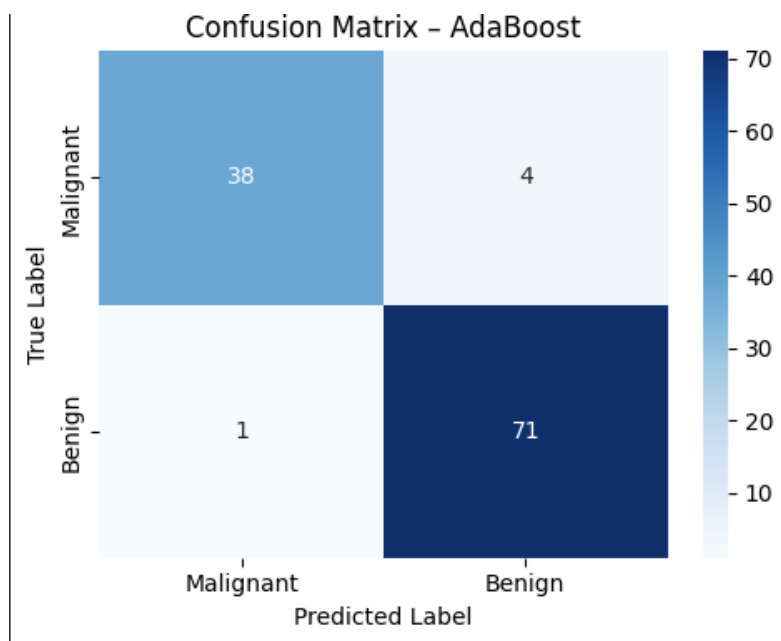


Figure 5: Confusion matrix for AdaBoost.

Inference: The AdaBoost matrix provides class-wise information about classification errors. It should be interpreted together with recall and precision rather than accuracy alone.

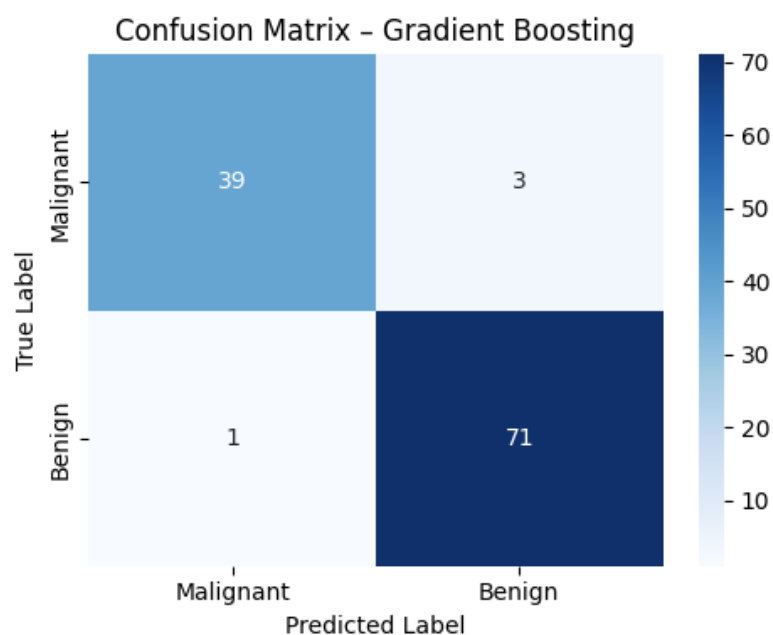


Figure 6: Confusion matrix for Gradient Boosting.

Inference: The matrix shows whether the model’s errors are concentrated in one class. This provides a useful diagnostic of class-specific behaviour.

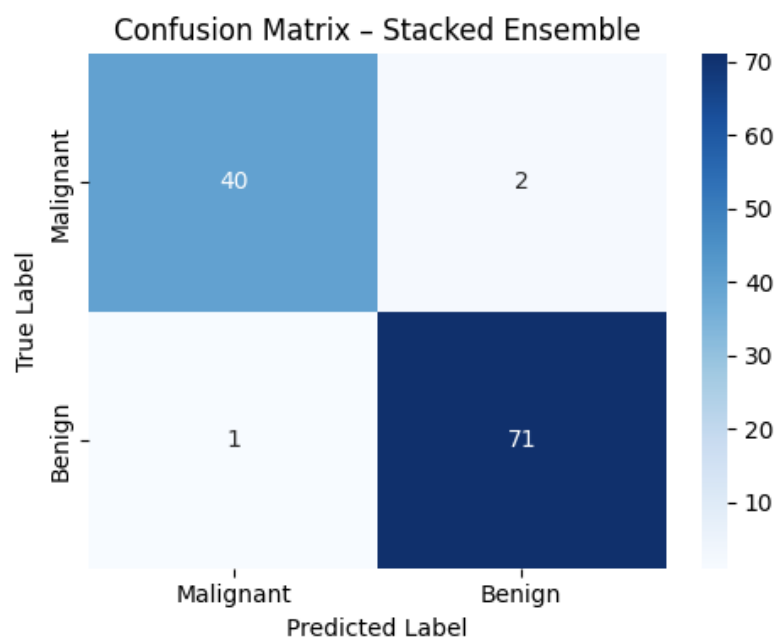


Figure 7: Confusion matrix for the Stacked Ensemble.

Inference: The stacked model’s matrix indicates its class-wise prediction behaviour on the held-out test set. Its results can be compared directly with the other ensemble models using the same test observations.

10.2 ROC Curve

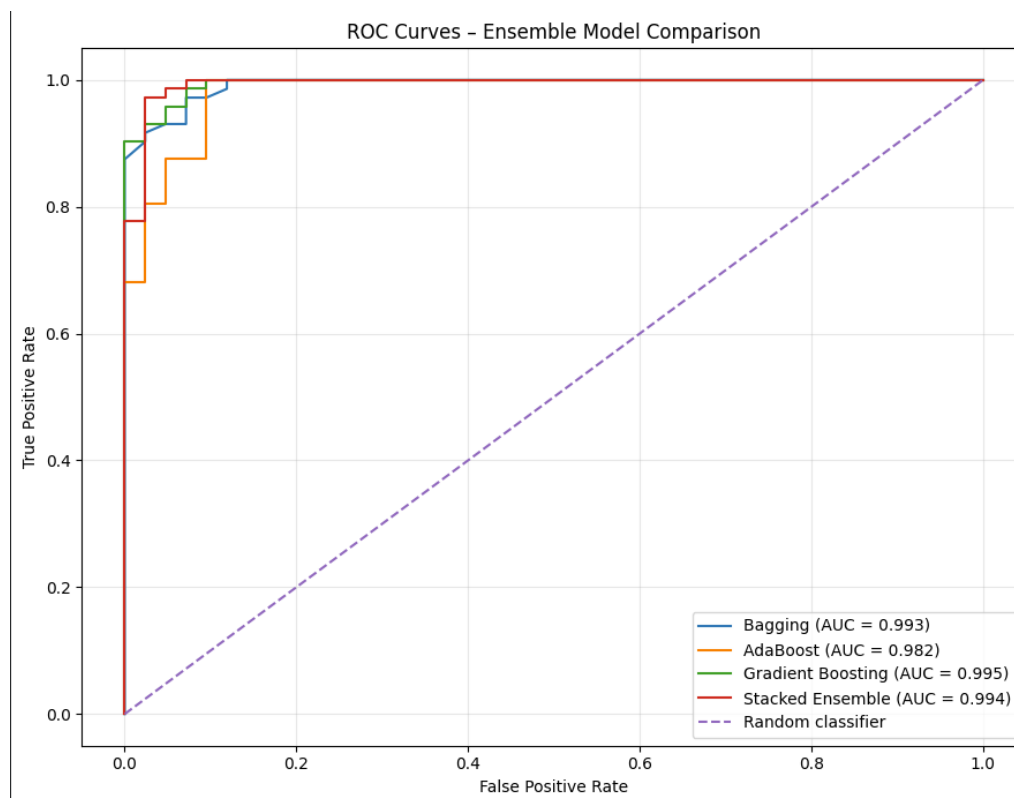


Figure 8: ROC curves and AUC values for the ensemble models.

Inference: The ROC curve describes the relationship between true-positive and false-positive rates over classification thresholds. The AUC summarizes discrimination across these thresholds. Higher observed AUC indicates better separation of the two classes on this test set, but it should be considered alongside the other metrics.

10.3 Precision-Recall Curve

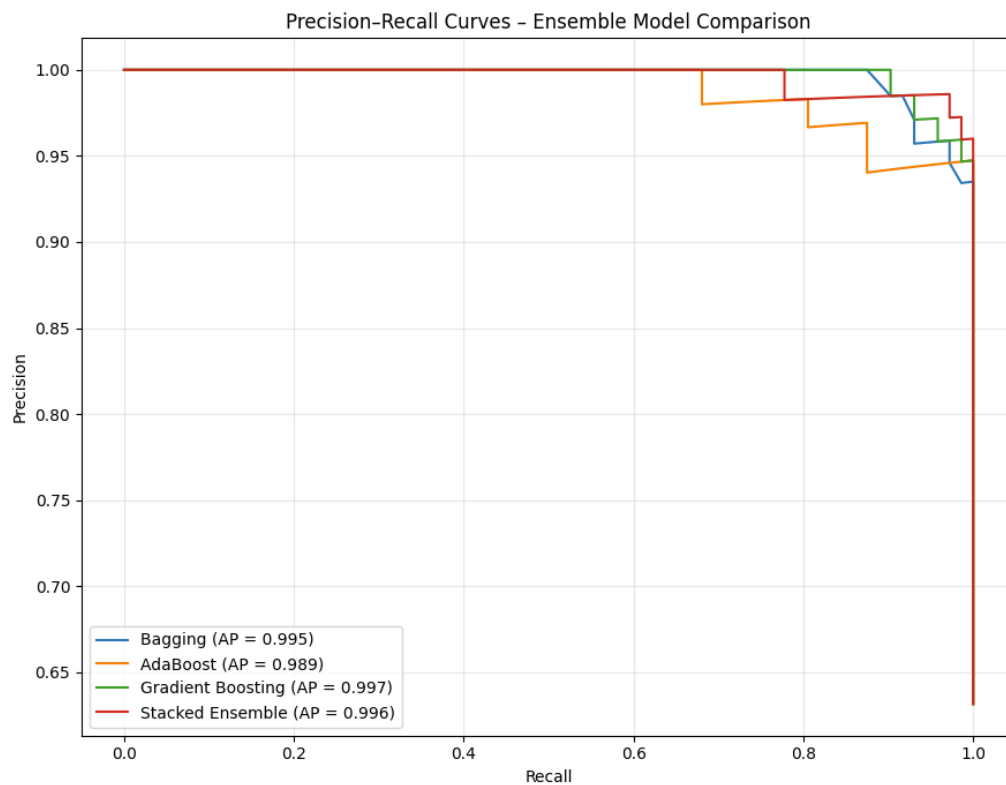


Figure 9: Precision-recall curves for the ensemble models.

Inference: Precision-recall curves emphasize the trade-off between precision and recall across thresholds. They provide complementary information to ROC curves, particularly when class distributions are not perfectly balanced.

10.4 Feature Importance

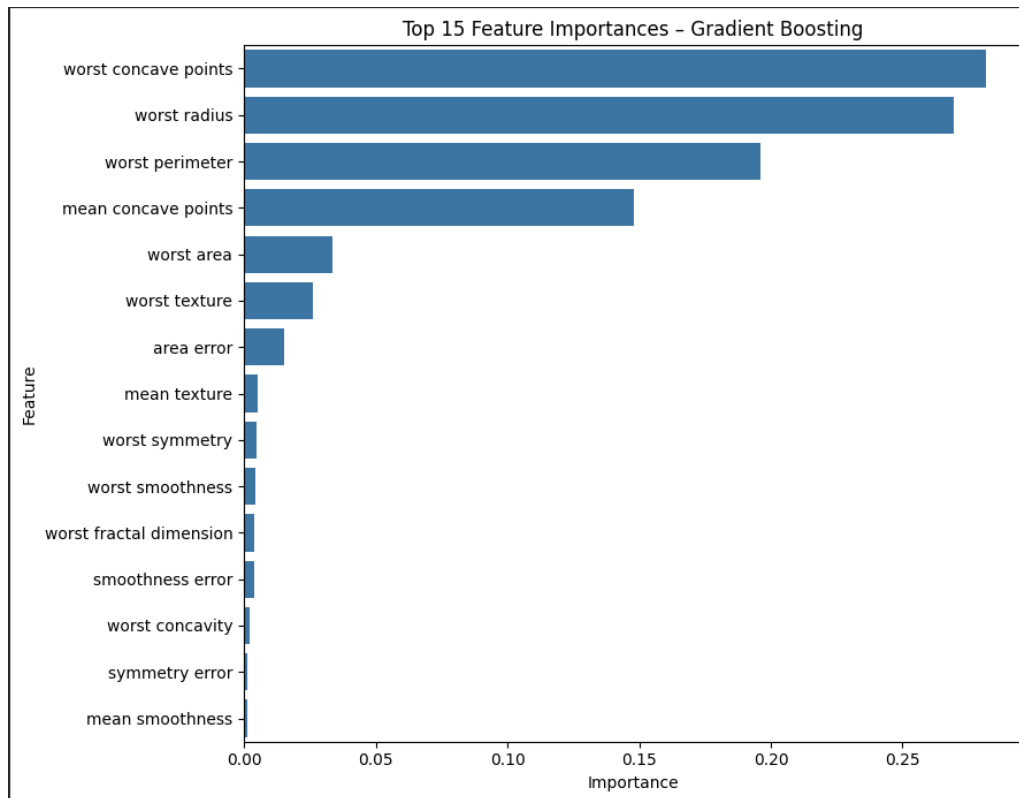


Figure 10: Top 15 feature importances from the selected Gradient Boosting model.

Inference: Feature importance identifies variables that contribute strongly to the Gradient Boosting model's decisions. It is a model-specific importance measure and should not be interpreted as a causal effect of a medical variable.

11 Discussion

11.1 Effect on Bias and Variance

Bagging primarily targets variance by averaging predictions from multiple bootstrap-trained estimators. Boosting adds learners sequentially to correct errors and can therefore reduce bias, while requiring careful control of learning rate and model complexity. Stacking attempts to exploit complementary strengths of heterogeneous learners through a meta-model.

11.2 Stability and Generalization

Five-fold cross-validation provides multiple training/validation views of the training data and allows the stability of model performance to be assessed using the mean and standard deviation of the scores. The final 20% test set is kept separate from hyperparameter selection to provide a held-out generalization estimate.

11.3 Strengths

- Multiple ensemble strategies are compared under a common experimental protocol.
- Hyperparameters are selected using stratified 5-fold cross-validation.
- Several complementary evaluation metrics and graphical diagnostics are used.
- The stacking ensemble uses heterogeneous base learners.

11.4 Limitations

- The final test estimate comes from one 20% split.
- Hyperparameter grids are finite and do not cover every possible configuration.
- Feature importance from a tree ensemble does not establish causality.
- Performance on this dataset does not by itself establish clinical suitability.

12 Observation Questions

1. How does Bagging reduce variance?

Bagging trains multiple models using bootstrap samples and aggregates their predictions. Since individual decision trees can be sensitive to small changes in training data, averaging across multiple trees reduces the effect of individual sampling fluctuations.

2. How does Boosting address model bias?

Boosting constructs models sequentially, with later learners concentrating on observations that previous learners predicted poorly. This progressive error correction can reduce systematic error and increase model flexibility.

3. Why does stacking benefit from heterogeneous models?

Different algorithms represent data relationships differently and therefore can make different errors. A meta-learner can use the predictions of SVM, Naive Bayes, and Decision Tree models to learn a combined prediction strategy.

4. Which ensemble method performed best and why?

The observed answer should be determined from the numerical test-set and cross-validation results generated by the notebook. The comparison should consider accuracy, precision, recall, F1-score, ROC-AUC, and the confusion matrices rather than selecting a model using a single metric.

13 Conclusion

Bagging, AdaBoost, Gradient Boosting, and a heterogeneous Stacked Ensemble were implemented on the Wisconsin Diagnostic Breast Cancer dataset. The experiment demonstrates three different approaches to ensemble learning: variance reduction through bootstrap aggregation, sequential error correction through boosting, and heterogeneous prediction combination through stacking. Five-fold cross-validation was used for model selection, while an independent 20% test set was used for final evaluation. The final experimental conclusion should be based on the numerical results produced by the notebook.

14 References

- [1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] Scikit-learn, “Ensemble methods,” <https://scikit-learn.org/stable/modules/ensemble.html>.
- [3] Scikit-learn, “BaggingClassifier,” <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingClassifier.html>.
- [4] Scikit-learn, “AdaBoostClassifier,” <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostClassifier.html>.
- [5] Scikit-learn, “GradientBoostingClassifier,” <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html>.
- [6] Scikit-learn, “StackingClassifier,” <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingClassifier.html>.
- [7] UCI Machine Learning Repository, “Breast Cancer Wisconsin (Diagnostic) Dataset,” <https://archive.ics.uci.edu/>.